

QR-NET Rainbow 1: An Optical Light-Based Mesh Network Protocol

Network Working Group
Request for Comments: QR-NET-1
Category: Experimental
Organization: QR-NET Project / ITCR
Date: May 2026

Abstract

This document specifies QR-NET Rainbow 1, a layered communication protocol that uses light-encoded QR images as a physical transmission medium. A transmitting node renders multicolor QR codes on a display; a receiving node reads them with a camera sensor. The protocol defines a physical layer (light modulation and multicolor encoding), a data-link layer (capability negotiation, 128-byte frames, selective retransmission), and a network layer (distance-vector mesh routing and anonymous virtual circuits via ephemeral identifiers).

The designation "Rainbow" refers to the multicolor encoding capability: up to 16 distinct colors per cell, yielding 4 bits per module and approximately 3.4 times the information density of classical 2-color QR codes and "1" corresponds with the expected functional version of the protocol.

Status of This Memo

This document is an experimental protocol specification produced as part of the Redes de Computadoras course at the Instituto Tecnológico de Costa Rica (ITCR), April–May 2026. It does not represent the output of an IETF Working Group and has no normative standing within the IETF standards process.

Distribution of this memo is unlimited.

Copyright Notice

Copyright © 2026 Ion Dolanescu. All rights reserved.

Table of Contents

- 1. Introduction
- 2. Conventions and Terminology
- 3. Protocol Architecture
- 4. Physical Layer (Layer 1)
 - 4.1. Grid Layout
 - 4.2. Format Information Band
 - 4.3. Color Encoding
 - 4.4. Color Calibration
 - 4.5. Capture Pipeline
 - 4.6. Optical Range Estimation
- 5. Data-Link Layer (Layer 2)
 - 5.1. HandshakeFrame Format
 - 5.2. Three-Way Handshake
 - 5.3. Handshake State Machine
 - 5.4. DataFrame Format
 - 5.5. Selective Retransmission (NACK)
 - 5.6. Session Termination (FIN)
 - 5.7. Medium Access Control
 - 5.8. Compression Negotiation
- 6. Network Layer (Layer 3)
 - 6.1. Packet Format
 - 6.2. Packet Type Definitions
 - 6.3. Node Addressing
 - 6.4. Distance-Vector Routing
 - 6.5. Anonymous Virtual Circuits
 - 6.6. HELLO and BYE Messages
 - 6.7. ROUTE_UPDATE Messages
- 7. Adapter Abstraction and Channel Selection
 - 7.1. NetworkAdapter Interface
 - 7.2. AdapterSelector
 - 7.3. Ethernet Adapter (TCP)
 - 7.4. WiFi Adapter (UDP Multicast)
- 8. Error Detection
- 9. Security Considerations
- 10. Acknowledgments
- 11. IANA Considerations
- 12. References
- 13. Authors' Address

1. Introduction

In environments where conventional telecommunications infrastructure is unavailable, restricted, or subject to surveillance, there is a need for communication channels that operate over unconventional physical media. QR-NET Rainbow 1 addresses this need by using the visible-light spectrum — specifically, camera-readable QR-style images rendered on a display — as a bidirectional data channel.

Two nodes equipped with a camera and a display can establish a QR-NET link by pointing their cameras at each other's screens. No physical cable, radio frequency allocation, or network infrastructure is required. The link can traverse a videoconference screen-share session, enabling communication across arbitrary geographic distances without exposing IP addresses or routing metadata to the underlying transport.

The design goals of QR-NET Rainbow 1 are:

- Operate over a visible-light optical channel using commodity camera and display hardware.
- Maximize information density through multicolor cell encoding (up to 16 colors, 4 bits per module).
- Provide reliable ordered delivery over an inherently lossy optical channel through a three-way handshake, per-frame checksums, and selective retransmission.
- Support multi-hop mesh networking with distance-vector routing and anonymous delivery via ephemeral virtual circuits.
- Remain implementable in software without privileged hardware access, using standard computer vision and networking libraries.

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

2. Conventions and Terminology

The following terms are used throughout this document:

Node: A device running the QR-NET Rainbow 1 protocol stack. A node has at least one camera (receiver) and one display (transmitter).

Module: The smallest addressable unit of a QR-NET grid. Equivalent to a "cell" in the grid image. Each module encodes 1, 2, 3, or 4 bits depending on the negotiated color depth.

Grid: The square array of modules that constitutes a single QR-NET frame image. The default grid is 68×68 modules total, comprising a 64×64 data area plus a 2-module silence zone on each side.

Frame: A complete encoded grid image representing one unit of transmission. Frame 0 is always a HandshakeFrame. Subsequent frames are DataFrames.

HandshakeFrame: The first frame transmitted in any QR-NET session. It carries capability advertisement fields and is used to negotiate session parameters through a three-way handshake.

DataFrame: A frame carrying user payload, a NACK control signal, or a FIN / FIN-ACK session termination signal.

Packet: The Layer 3 protocol data unit. A Packet is carried in the payload of one or more DataFrames. It includes source and destination Node IDs, a TTL counter, and an optional circuit identifier.

Circuit: An ephemeral virtual path between two nodes identified by a 32-bit `circuit_id`. Intermediate nodes forward packets by `circuit_id` without knowing the origin or final destination.

Color Depth (CD): The number of distinct colors used per module. Negotiated in the HandshakeFrame. Valid values: 2, 4, 8, or 16.

Silence Zone (SZ): A border of blank (white) modules surrounding the grid. QR-NET uses a 2-module silence zone on each side.

Finder Pattern (FP): A 7×7 module pattern placed in three corners of the grid, used by the receiver to locate and align the grid image.

Calibration Patch (CAL): A 4×4 module region carrying reference color samples, used by the receiver to calibrate color classification under varying lighting conditions.

Format Information Band (FIB): A single-module-wide band adjacent to the finder patterns that encodes grid metadata: payload length and color depth.

MAC Address: A 6-byte node identifier analogous to an IEEE 802 MAC address, used at the data-link layer for frame addressing.

Node ID: A 16-byte ephemeral identifier used at the network layer. Regenerated at each session start to support anonymity.

TTL: Time To Live. An 8-bit counter decremented by each forwarding node. A packet with TTL equal to zero is silently discarded.

BEB: Binary Exponential Backoff. The medium access control algorithm used to reduce collision probability on shared optical channels.

NACK: Negative Acknowledgment. A control frame sent by the receiver to request retransmission of a specific DataFrame.

FIN: A DataFrame variant signaling end of transmission.

FIN-ACK: A DataFrame variant acknowledging a received FIN.

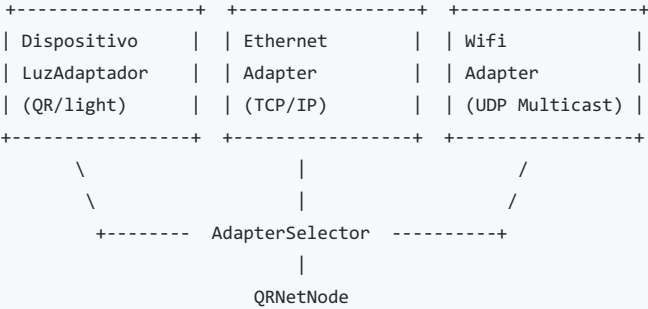
SHA-128: A truncated SHA-256 digest reduced to the first 128 bits (16 bytes), used for file integrity verification.

3. Protocol Architecture

QR-NET Rainbow 1 maps to the OSI reference model as follows:

+-----+	
Layer 7 (Application)	
Anonymous microblogging / chat (AnonChatApp)	
Clearnet bridge: IRC server / NNTP relay	
+-----+	
Layer 3 (Network) -- remote-QR-net	
Distance-vector mesh routing (HELLO, ROUTE_UPDATE, BYE)	
Anonymous virtual circuits (ephemeral circuit_id)	
+-----+	
Layer 2 (Data Link)	
HandshakeFrame: three-way capability negotiation	
DataFrame: max 128 bytes, CRC-16, sequence numbering	
Selective retransmission (NACK), FIN / FIN-ACK	
Binary Exponential Backoff (MAC sublayer)	
+-----+	
Layer 1 (Physical)	
66x66 module custom grid with 2-module silence zone	
Multicolor encoding: 2 / 4 / 8 / 16 colors per module	
Finder patterns + calibration patch + format band	
FIFO producer-consumer capture pipeline	
+-----+	

The protocol is channel-agnostic above Layer 1. An AdapterSelector component chooses among available transport channels at runtime:



4. Physical Layer (Layer 1)

4.1. Grid Layout

Each QR-NET frame is rendered as a square image of modules. The default grid contains a 64×64 data area surrounded by a 2-module silence zone on all four sides, producing a 68×68 module total image.

Full grid layout (not to scale; SZ = silence zone module):

SZ SZ SZ SZ SZ SZ SZ SZ SZ SZ SZ SZ SZ ... SZ SZ SZ SZ SZ SZ SZ
SZ +-----+ SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | F | SZ
SZ | I | SZ
SZ | B DATA + ECC PAYLOAD . . | SZ
SZ | . (variable content) . . | SZ
SZ | | SZ
SZ | [CAL 4x4] | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 . . . FP7 FP7 | SZ
SZ | FP7 FP7 FP7 FP7 FP7 FP7 FP7 . . . FP7 FP7 | SZ
SZ +-----+ SZ
SZ SZ SZ SZ SZ SZ SZ SZ SZ SZ SZ SZ SZ ... SZ SZ SZ SZ SZ SZ SZ

Legend:

Symbol	Description
SZ	Silence Zone module (white, not encoded)
FP7	Finder Pattern module (7×7 pattern, 3 corners)
FIB	Format Information Band (single-module column)
CAL	Calibration Patch (4×4 modules)
.	Data/ECC payload module

Module accounting (default 64×64 data area):

Region	Modules
Total modules in data area (64×64)	4,096
Three finder patterns (3 × 7×7)	147
Calibration patch (4×4)	16
Format information band	32
Net payload modules available	3,901

Capacity at each color depth:

Color Depth	Colors	Bits/mod	Raw bytes/frame
CD=00	2	1	487
CD=01	4	2	975
CD=10	8	3	1,463

CD=11	16	4	Raw bytes/frame	1,950
Color Depth	Colors	Bits/mod		

The silence zone is reduced from the 4-module margin of ISO/IEC 18004 to 2 modules because:

- (a) Both nodes negotiate the exact grid dimensions in the handshake (GRID field), so the receiver knows precisely where the grid boundary is.
- (b) The receiving camera operates in a controlled environment (neutral background, fixed framing), providing sufficient contrast for the edge detector at 2-module margins.
- (c) Reducing the silence zone from 4 to 2 modules recovers additional payload space along each edge.

Grid size is negotiated via the GRID field (6 bits) in the HandshakeFrame. The value is the data module count per side divided by 8. A value of 0x08 specifies the default 64x64 data area. Implementations MUST support GRID=0x08. Support for other values is OPTIONAL.

4.2. Format Information Band (FIB)

The Format Information Band is a single-module-wide column adjacent to the left finder pattern, spanning rows 8 through row (8 + FIB_LEN - 1) of the data area. It encodes grid metadata that the receiver needs before decoding the payload.

FIB encoding (32 modules, big-endian, 1 bit per module using the 2-color palette regardless of the session color depth):

Bit 0 - 3	: PAYLOAD_LEN high nibble (4 bits)
Bit 4 - 7	: PAYLOAD_LEN low nibble (4 bits)
Bit 8 - 9	: CD (Color Depth, 2 bits)
Bit 10 - 15	: GRID (grid size, 6 bits)
Bit 16 - 31	: CRC-16 of bits 0-15 (16 bits)

PAYLOAD_LEN is the number of payload bytes encoded in this frame's payload modules (excluding header overhead carried at Layer 2).

The FIB MUST always use 2-color (black/white) encoding so that the receiver can parse it before knowing the negotiated color depth. The receiver reads the FIB first, then uses the CD field therein to decode the payload modules.

4.3. Color Encoding

QR-NET Rainbow 1 extends classical black/white QR encoding to use up to 16 distinct colors. Each module carries an n-bit symbol where $n = \log_2(\text{num_colors})$.

Color depth coding table:

CD field	Colors	Bits/module
00	2	1
01	4	2
10	8	3
11	16	4

Palette definitions by color depth:

CD=00 (2 colors):

Index	Name	RGB
0	Black	(0, 0, 0)
1	White	(255, 255, 255)

CD=01 (4 colors):

Index	Name	RGB
0	Black	(0, 0, 0)
1	White	(255, 255, 255)
2	Red	(216, 0, 0)

3	Cyan	(0, 216, 216)
Index	Name	RGB

CD=10 (8 colors):

Index	Name	RGB
0	Black	(0, 0, 0)
1	White	(255, 255, 255)
2	Red	(216, 0, 0)
3	Cyan	(0, 216, 216)
4	Green	(0, 216, 0)
5	Magenta	(216, 0, 216)
6	Blue	(0, 0, 216)
7	Yellow	(216, 216, 0)

CD=11 (16 colors, HSV-uniform palette):

Index	Name	H	S	V
0	Black	0	0.0	0.00
1	White	0	0.0	1.00
2	Red	0	1.0	0.85
3	Orange	30	1.0	0.85
4	Yellow	60	1.0	0.85
5	Chartreuse	90	1.0	0.85
6	Green	120	1.0	0.85
7	Spring	150	1.0	0.85
8	Cyan	180	1.0	0.85
9	Azure	210	1.0	0.85
10	Blue	240	1.0	0.85
11	Violet	270	1.0	0.85
12	Magenta	300	1.0	0.85
13	Rose	330	1.0	0.85
14	Light Gray	0	0.0	0.50
15	Dark Gray	0	0.0	0.25

The 16-color palette is derived by sampling the HSV color space at uniform 30-degree hue intervals with fixed saturation S=1.0 and value V=0.85, plus black, white, light gray, and dark gray.

Encoding procedure:

1. Take the payload byte sequence to be encoded.
2. Split into n-bit symbols (e.g., 4-bit nibbles for CD=11).
3. Map each symbol (0 to num_colors - 1) to its palette RGB.
4. Render the mapped color as a filled square covering one module region in the output image.

Decoding procedure:

1. Read the average RGB value of each module region.
2. Apply the calibration offset (Section 4.4).
3. Classify the corrected RGB to the nearest palette entry using minimum Euclidean distance in RGB space.
4. The palette index is the decoded n-bit symbol.

4.4. Color Calibration

Lighting conditions at the receiver may shift observed colors uniformly from their transmitted values. The Calibration Patch (CAL) addresses this by carrying a fixed, known reference.

The CAL patch occupies a 4x4 module region. Its position is fixed at grid coordinates (row=56, col=56) within the 64x64 data area (0-indexed from the top-left of the data region). The CAL patch encodes the first `num_colors` palette entries in sequential index order, repeated as needed to fill 16 modules.

Calibration procedure (MUST be performed per received frame):

1. Locate the CAL patch at the fixed coordinates.
2. For each of the `num_colors` reference modules, measure the observed RGB centroid by averaging the pixel values within the module region.
3. For each palette entry `i`, compute the per-channel offset:

```
delta_R[i] = expected_R[i] - observed_R[i]
delta_G[i] = expected_G[i] - observed_G[i]
delta_B[i] = expected_B[i] - observed_B[i]
```

4. Compute the global mean offset:

```
delta_R = mean(delta_R[0..n-1])
delta_G = mean(delta_G[0..n-1])
delta_B = mean(delta_B[0..n-1])
```

5. Apply the global offset to all subsequent module reads within the same frame before nearest-palette classification.

Calibration MUST be performed once per frame. Calibration state MUST NOT persist across frames.

4.5. Capture Pipeline

The QR-NET physical layer receiver uses a two-thread producer-consumer pipeline to decouple capture from decoding:

Thread PRODUCER	Thread CONSUMER
-----	-----
loop:	loop:
<code>img <- camera.capture()</code>	<code>img <- fifo.get(timeout=1s)</code>
<code>if detect_change(img):</code>	<code>raw <- grid.decode(img)</code>
<code> fifo.put(img)</code>	<code>calibrate_colors(raw)</code>
<code>sleep(interval_ms)</code>	<code>emit_to_layer2(raw)</code>
<code> </code>	<code> ^</code>
+----- FIFO Queue -----+	
(thread-safe)	
maxsize = 32 frames	

`detect_change()` computes the mean absolute pixel difference between the current grayscale frame and the previous one. A frame is enqueued only if the difference exceeds a threshold (default: 8 gray levels). This prevents the consumer from processing duplicate frames.

Queue behavior under load:

Condition	Behavior
Consumer faster than cam	Queue empty; nominal operation
Consumer slower than cam	Queue grows; frames buffered
Queue full (maxsize=32)	Configurable: drop-oldest/block
Duplicate frame detected	Discarded before enqueue

The PRODUCER thread MUST NOT call `grid.decode()` directly. The CONSUMER thread MUST NOT call `camera.capture()` directly. This separation ensures that a slow decoder does not cause the camera capture loop to stall.

4.6. Optical Range Estimation

The maximum reliable transmission range depends on the display size, the physical grid dimensions, and the camera sensor resolution. For reliable color classification each module **MUST** subtend at least 3 camera pixels along each axis; a minimum of 4 pixels per module is **RECOMMENDED**.

Estimated operational ranges by hardware configuration:

Scenario	Display	Camera	Range
Laptop to laptop	15" FHD	Webcam 1080p	0.8–1.5 m
Monitor + webcam	27" 4K	USB 1080p60	1.5–3.0 m
Screen share	Virtual	Any FHD+	Unlimited*
Projector wall	120" projected	HD camera	5–10 m

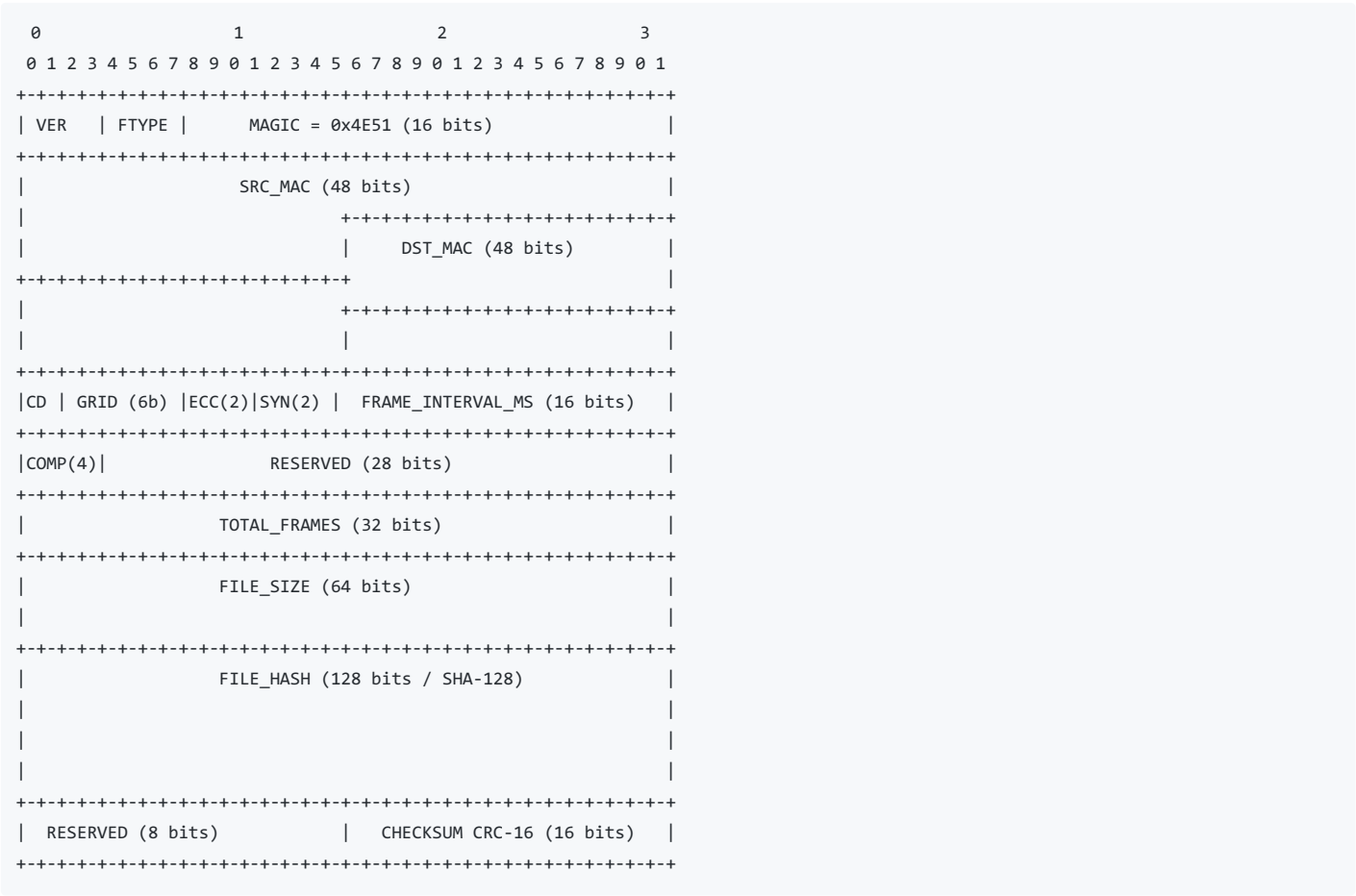
(*) Screen share routes light through video conferencing infrastructure; actual throughput depends on the video codec quality and frame rate imposed by the platform.

5. Data-Link Layer (Layer 2)

5.1. HandshakeFrame Format

The HandshakeFrame is Frame 0 of every QR-NET session. It carries capability advertisement and is used to negotiate session parameters. Its `frame_type` field identifies the phase of the three-way handshake.

HandshakeFrame wire format (big-endian):



Total HandshakeFrame size: **48 bytes**.

Field descriptions:

VER (4 bits): Protocol version. This specification defines version `0x1`. Implementations **MUST** discard frames with an unrecognized VER value.

FTYPE (4 bits): Handshake phase identifier:

- 0x0 SYN – Initiator capability advertisement
- 0x1 SYN-ACK – Responder advertisement and agreement
- 0x2 ACK – Initiator confirmation of negotiated values

MAGIC (16 bits): Fixed value 0x4E51 (ASCII "NQ", mnemonic for "QR-Net"). Implementations MUST discard HandshakeFrames with an incorrect MAGIC value.

SRC_MAC (48 bits): 6-byte MAC address of the transmitting node.

DST_MAC (48 bits): 6-byte MAC address of the intended recipient. All-ones (0xFF repeated 6 times) indicates broadcast.

CD (2 bits): Color Depth. Maximum capability offered in SYN and SYN-ACK; the agreed value carried in ACK:

- 00 – 2 colors (1 bit/module)
- 01 – 4 colors (2 bits/module)
- 10 – 8 colors (3 bits/module)
- 11 – 16 colors (4 bits/module)

GRID (6 bits): Grid size in multiples of 8 data modules per side. 0x08 = 64×64 data modules (default and REQUIRED minimum).

ECC (2 bits): Error Correction Code level:

- 00 – None
- 01 – Low (~7% of modules used for ECC)
- 10 – Medium (~15% of modules used for ECC)
- 11 – High (~30% of modules used for ECC)

SYN (2 bits): Synchronization method:

- 00 – Timer-based (fixed FRAME_INTERVAL_MS)
- 01 – Visual difference detection
- 10 – QR grid edge tick
- 11 – Hybrid (visual diff with timer fallback)

FRAME_INTERVAL_MS (16 bits): Time in milliseconds between consecutive frames. The negotiated value is the maximum of the two nodes' proposals (the slower node determines the session rate).

COMP (4 bits): Compression algorithm for DataFrame payload:

- 0x0 – None (no compression)
- 0x1 – Zstandard (zstd) level 3 [RECOMMENDED]
- 0x2 – LZ4
- 0x3 – Brotli
- 0x4–0xF – Reserved for future use

TOTAL_FRAMES (32 bits): Number of DataFrames comprising the complete transmission. MUST be set to 0 in SYN (unknown at initiation time).

FILE_SIZE (64 bits): Original (pre-compression) file size in bytes. MUST be set to 0 in SYN.

FILE_HASH (128 bits): SHA-128 of the original uncompressed file content. Used by the receiver to verify integrity after reassembly and decompression. MUST be set to all-zeros in SYN.

RESERVED: MUST be set to zero on transmission. MUST be ignored on receipt.

CHECKSUM (16 bits): CRC-16/CCITT-FALSE computed over all preceding bytes of the HandshakeFrame. Implementations MUST discard frames with an invalid checksum.

Capability negotiation rules:

For CD, ECC, and COMP: the agreed value in ACK MUST be the minimum of the initiator's SYN value and the responder's SYN-ACK value.

For FRAME_INTERVAL_MS : the agreed value in ACK MUST be the maximum of the two proposed values.

This ensures the session uses only parameters that both nodes support.

5.2. Three-Way Handshake

QR-NET uses a three-way handshake to establish session parameters before data transmission. Roles are asymmetric: one node is the Initiator (sends SYN); the other is the Responder (waits for SYN, replies with SYN-ACK).

Message sequence:

INITIATOR (A)	RESPONDER (B)
-- HandshakeFrame (FTYPE=SYN) ----->	
CD=11, INTERVAL=150, COMP=zstd	
<- HandshakeFrame (FTYPE=SYN-ACK) ---	
CD=10, INTERVAL=200, COMP=zstd	
-- HandshakeFrame (FTYPE=ACK) ----->	
CD=10, INTERVAL=200, COMP=zstd	
TOTAL_FRAMES=N, FILE_SIZE=S	
FILE_HASH=H	
-- DataFrame (seq=1, DATA) ----->	
-- DataFrame (seq=2, DATA) ----->	
...	
-- DataFrame (seq=N, DATA) ----->	
<- DataFrame (FTYPE=NACK, seq=k) ----	(if frame k lost)
-- DataFrame (seq=k, DATA) ----->	(retransmission)
-- DataFrame (FTYPE=FIN, seq=N+1) -->	
<- DataFrame (FTYPE=FIN-ACK) -----	

Timeout and retransmission:

The Initiator MUST retransmit SYN up to 3 times with a 10-second timeout between attempts before declaring the session failed.

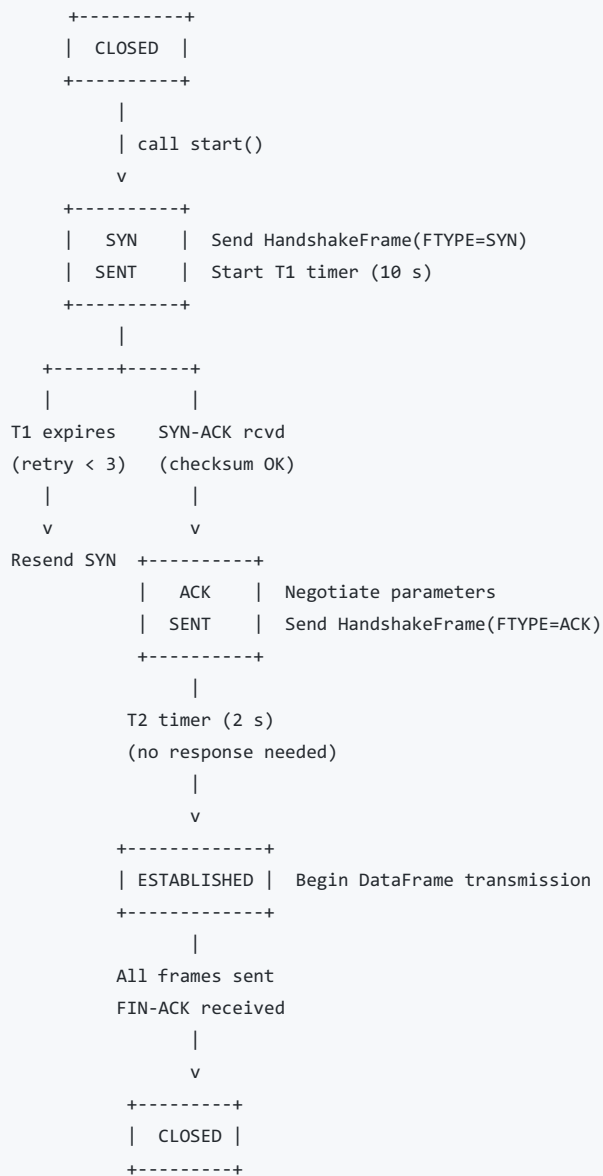
The Responder MUST retransmit SYN-ACK up to 3 times with a 10-second timeout if no ACK is received.

An implementation that fails to complete the handshake MUST log an error and terminate the session.

5.3. Handshake State Machine

The following state diagram describes the Initiator side. The Responder side is symmetric (`CLOSED` → `LISTEN` → `SYN-RCVD` → `ESTABLISHED`).

Initiator state machine:



Error transitions:

From any state: If T1 expires 3 times → go to CLOSED, report `HANDSHAKE_TIMEOUT`.

From any state: If a HandshakeFrame with invalid MAGIC or invalid checksum is received → discard and stay in current state (do not reset timer).

5.4. DataFrame Format

DataFrames carry user payload, NACK control signals, and FIN / FIN-ACK session termination signals.

A DataFrame MUST NOT exceed 128 bytes in total serialized size.

DataFrame wire format (big-endian):



Field descriptions:

VER (4 bits): Protocol version. MUST be 0x1

FTYPE (4 bits): Frame type:

- 0x0 DATA – User payload
- 0x1 NACK – Negative acknowledgment
- 0x2 FIN – End-of-transmission signal
- 0x3 FIN-ACK – End-of-transmission acknowledgment

SRC_MAC (48 bits): 6-byte MAC address of the transmitting node.

DST_MAC (48 bits): 6-byte MAC address of the intended recipient.

SEQ_NUM (16 bits): Sequence number, starting at 1 for the first DataFrame. Frame 0 is always a HandshakeFrame. NACK frames carry the sequence number of the missing DATA frame. FIN frames carry the sequence number of the next frame that would have followed the last DATA frame.

PAYLOAD_LEN (8 bits): Number of bytes in the PAYLOAD field.

PAYLOAD (variable):

- DATA: compressed user data bytes (up to 110 bytes).
- NACK: the first 2 bytes encode the missing `SEQ_NUM` as a big-endian unsigned 16-bit integer.
- FIN: empty (`PAYLOAD_LEN = 0`).
- FIN-ACK: empty (`PAYLOAD_LEN = 0`).

CHECKSUM (16 bits): CRC-16/CCITT-FALSE over all preceding DataFrame bytes. Implementations **MUST** discard frames with an invalid checksum and **MUST NOT** pass them to higher layers.

QR-NET uses negative acknowledgment for loss recovery. The receiver detects a missing frame when it observes a gap in sequence numbers or when a frame fails its checksum.

NACK procedure:

1. Receiver detects that frame with SEQ_NUM=k is absent (gap observed, or checksum failure).
2. Receiver constructs and transmits a NACK DataFrame:
 - o FTYPE = 0x1 (NACK)
 - o SEQ_NUM = k
 - o PAYLOAD = big-endian uint16 encoding of k
 - o PAYLOAD_LEN = 2
3. Transmitter receives the NACK, retrieves DataFrame k from its sent-frame cache, and retransmits it unchanged.
4. If the transmitter no longer holds DataFrame k in its cache, it SHOULD transmit a FIN to terminate the session, indicating that recovery is not possible.

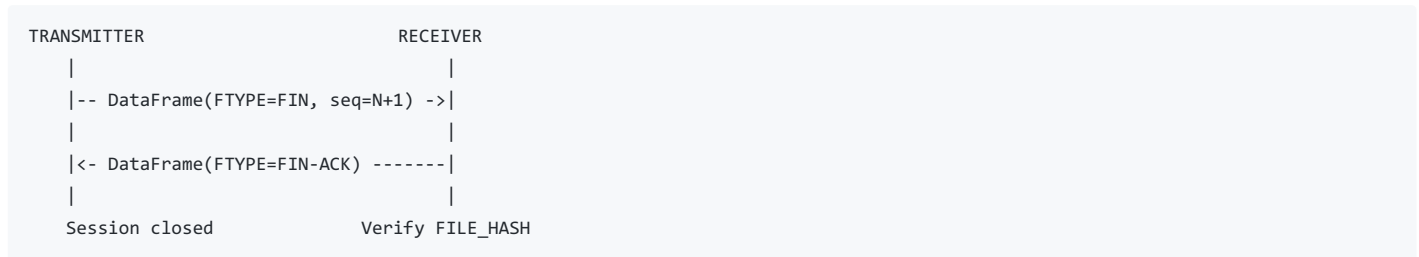
The transmitter MUST maintain a cache of all transmitted DATA frames for the duration of the session.

This specification does not define a window size for outstanding NACKs. Implementations MAY impose a limit.

5.6. Session Termination (FIN)

End-of-transmission is signaled by a FIN DataFrame.

Termination exchange:



The transmitter MUST wait up to 10 seconds for FIN-ACK. If no FIN-ACK is received, the transmitter MAY treat the session as best-effort complete, because the optical channel may be physically unidirectional in some hardware configurations.

Upon receiving FIN the receiver MUST:

1. Transmit a FIN-ACK DataFrame.
2. Reassemble all received DATA frame payloads in `SEQ_NUM` order.
3. Decompress the reassembled data using the algorithm agreed in the handshake `COMP` field.
4. Compute SHA-128 of the decompressed data.
5. Compare the result to `FILE_HASH` from the HandshakeFrame. If they differ, the receiver MUST notify the operator of a data integrity failure.

5.7. Medium Access Control

The optical channel is shared among all nodes with a line of sight to the same display. QR-NET uses Binary Exponential Backoff (BEB), adapted from IEEE 802.3 CSMA/CD, to reduce collision probability among concurrent transmitters.

Collision detection: A collision is inferred when the adapter's `send()` call returns `False`, indicating that the channel was busy or that simultaneous transmission was detected.

Backoff algorithm:

Constants:

BASE	= 10 ms	(minimum slot duration)
MAX	= 320 ms	(maximum backoff delay)
WIN	= 50 ms	(collision recency window)
LVLMAX	= 5	(maximum backoff level, giving $2^5 = 32$ slots)

Per-transmission procedure:

```
if (now - last_collision_time) > WIN:
    level <- 0
    transmit immediately
else:
    slots <- uniform_random(0, 2^min(level, LVLMAX) - 1)
    delay <- min(slots * BASE, MAX)
    level <- min(level + 1, LVLMAX)
    wait delay milliseconds
    transmit

After transmission:
if send() returns False:
    last_collision_time <- now
```

All transmission paths (unicast and broadcast) within a single node MUST share the same backoff state and lock to prevent intra-node collisions between concurrent threads.

5.8. Compression Negotiation

Payload data SHOULD be compressed before fragmentation into DataFrames. The compression algorithm is negotiated via the `COMP` field in the HandshakeFrame.

Recommended algorithm: Zstandard (zstd) level 3.

Throughput estimation (CD=11, 10 fps, default grid):

Mode	Bytes/frame	fps	Effective KB/s
No compression	1,950	10	~19
zstd, binary (~40% reduction)	1,950	10	~32
zstd, text (~70% reduction)	1,950	10	~65

Negotiation rule: If the receiver does not support the algorithm proposed by the initiator, it MUST include the best algorithm it supports in the COMP field of its SYN-ACK. The initiator MUST accept and use that algorithm for the session.

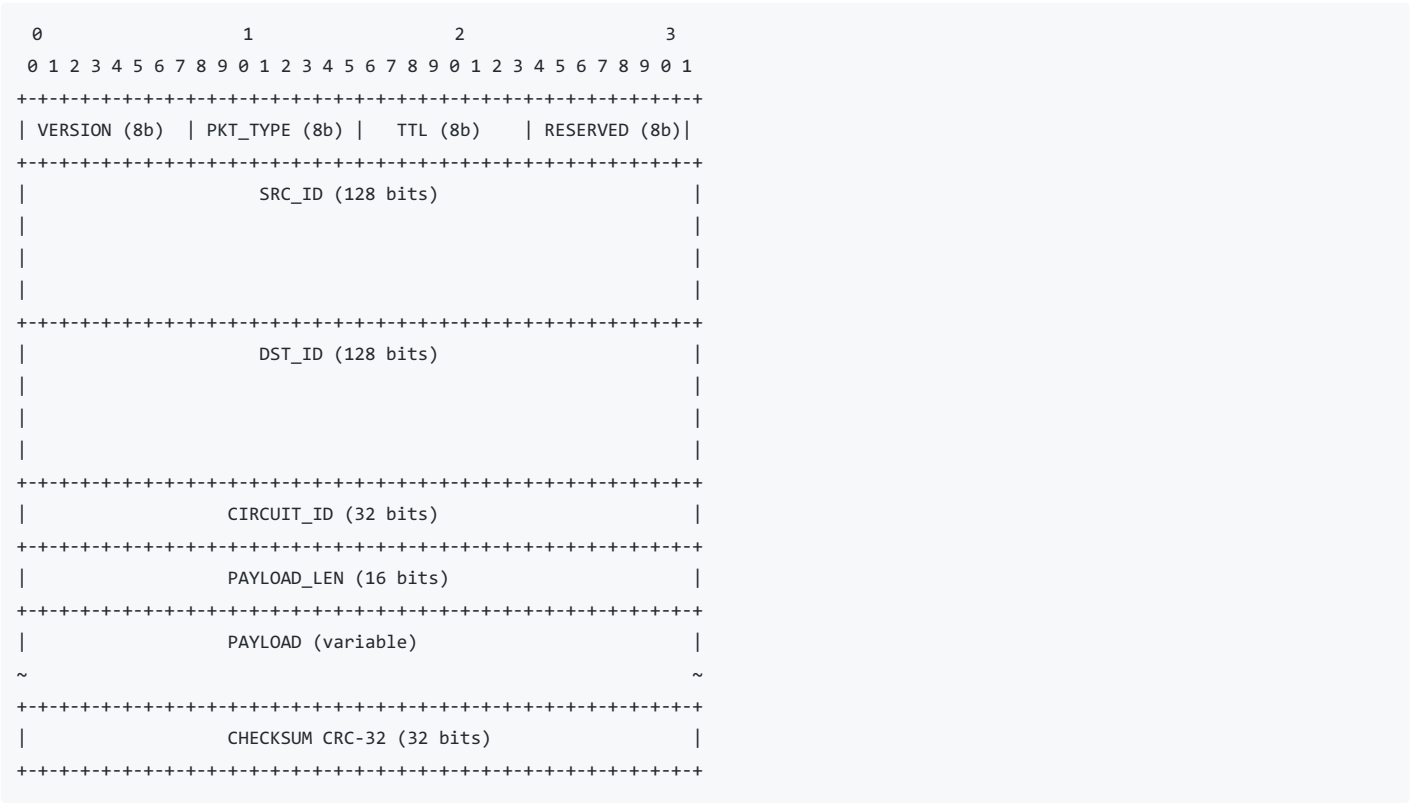
COMP=0x0 (no compression) MUST be supported by all implementations as a mandatory baseline.

6. Network Layer (Layer 3)

6.1. Packet Format

The network layer Packet is the Layer 3 PDU. It is carried in the PAYLOAD field of one or more DataFrames. All fields are big-endian.

Packet wire format:



Header size (without payload or checksum): 42 bytes. With checksum field: 46 bytes.

Field descriptions:

VERSION (8 bits): Protocol version. This specification defines 0x1.

PKT_TYPE (8 bits): Packet purpose. See Section 6.2.

TTL (8 bits): Time To Live. MUST be set to 16 at the originating node. Each forwarding node MUST decrement TTL by 1 before forwarding. A node that receives a packet with TTL=0 MUST silently discard it.

RESERVED (8 bits): MUST be zero on transmission; ignored on receipt.

SRC_ID (128 bits): Ephemeral 16-byte Node ID of the originator. For packets with CIRCUIT_ID != 0 (anonymous packets), this field carries the CIRCUIT_ID left-padded with 12 zero bytes, hiding the true origin from intermediate nodes.

DST_ID (128 bits): 16-byte Node ID of the destination. The all-ones value (0xFF repeated 16 times) is the network broadcast address.

CIRCUIT_ID (32 bits): Virtual circuit identifier for anonymous delivery. Zero indicates a non-anonymous packet.

PAYLOAD_LEN (16 bits): Length in bytes of the PAYLOAD field.

PAYLOAD (variable): Content depends on `PKT_TYPE` (see Section 6.2).

CHECKSUM (32 bits): CRC-32 over all preceding packet bytes. Receivers MUST discard packets with invalid checksums and MUST NOT use any field from such a packet to construct a reply.

6.2. Packet Type Definitions

PKT_TYPE value	Name	PAYLOAD content
<code>0x00</code>	DATA	User or application data
<code>0x01</code>	HELLO	Node ID of sender (16 bytes)
<code>0x02</code>	BYE	Empty
<code>0x03</code>	CIRCUIT_SETUP	Empty
<code>0x04</code>	CIRCUIT_TEARDOWN	Empty
<code>0x05</code>	ROUTE_UPDATE	JSON array (see Section 6.7)

All other `PKT_TYPE` values are reserved. Implementations MUST discard packets with unrecognized `PKT_TYPE` values.

6.3. Node Addressing

QR-NET nodes are identified at the network layer by 16-byte ephemeral Node IDs. These identifiers:

- MUST be generated using a cryptographically secure pseudo-random number generator at session start.
- MUST NOT be derived from hardware identifiers, MAC addresses, or IP addresses, to prevent correlation.
- MUST be regenerated at each new network session start, ensuring that two sessions from the same physical device cannot be linked by a passive observer who only observes Layer 3 headers.

The all-ones Node ID (`0xFF` repeated 16 times) is the network broadcast address. No node may use this as its Node ID.

A node MUST NOT reuse a previously assigned Node ID in a new session.

6.4. Distance-Vector Routing

QR-NET implements a simplified distance-vector routing protocol. Each node maintains a routing table mapping destination Node IDs to (`next-hop MAC`, `cost`) entries.

Routing table entry fields:

Field	Description
<code>dst_id</code>	16-byte destination Node ID
<code>next_hop_mac</code>	6-byte link-layer address of next hop
<code>cost</code>	Integer hop count to destination
<code>updated_at</code>	Monotonic timestamp of last update

Route installation policy:

A route MUST be installed or updated if:

- (a) No existing entry exists for `dst_id`, OR
- (b) The received cost is less than the current stored cost.

A route MUST update its timestamp (keeping the same cost) if:

- (c) The received cost equals the current stored cost. (This prevents premature expiration of equal-cost routes.)

A route MUST NOT be installed if the received cost is strictly greater than the current stored cost. This implements the "best cost wins" policy.

Route propagation events:

A node MUST broadcast a ROUTE_UPDATE in each of the following circumstances:

- (a) After successfully processing a HELLO from a new neighbor.
- (b) After removing a route due to a received BYE.
- (c) After discover_peers() returns one or more new nodes.
- (d) Periodically at ROUTE_UPDATE_INTERVAL (default: 45 s).

ROUTE_UPDATE payload format (JSON):

```
[
  {"dst_id": <32 lowercase hex chars>, "cost": <int>},
  ...
]
```

Each entry represents a destination reachable via this node. Entries with cost >= 16 (TTL_DEFAULT) MUST NOT be included (simplified poison reverse).

A receiving node MUST add 1 to each received cost before installing the route.

ROUTE_UPDATE packets carry TTL=1 and MUST NOT be re-broadcast by receiving nodes.

Route expiration: Routes not refreshed within STALE_ROUTE_S (default: 90 seconds) MUST be purged from the routing table.

Forwarding algorithm:

Upon receiving a packet, a node performs the following steps in order:

1. Compute CRC-32 over the packet. If invalid, discard silently and stop.
2. If DST_ID equals this node's Node ID, deliver the PAYLOAD to the local application buffer and stop.
3. If TTL equals 0, discard silently and stop.
4. Decrement TTL by 1.
5. If CIRCUIT_ID is non-zero, forward per Section 6.5.
6. If DST_ID is the broadcast address, flood: send the packet out all available adapters.
7. Look up DST_ID in the routing table. If found: forward to next_hop_mac . If not found: discard silently.

6.5. Anonymous Virtual Circuits

QR-NET provides network-layer anonymity through ephemeral virtual circuits. A CIRCUIT_ID (32-bit random value) associates an anonymous communication stream with a pair of (prev_hop_mac, next_hop_mac) entries at each intermediate node. No intermediate node has enough information to determine the full path.

Circuit table entry at each node:

Field	Description
circuit_id	uint32 identifier
prev_hop_mac	6-byte MAC of node from which frames arrive (None at originating node)
next_hop_mac	6-byte MAC of node to which frames forward (None at terminating node)

Circuit setup:



Originator behavior:

1. Generates a random 32-bit CIRCUIT_ID.
2. Stores (circuit_id, prev=None, next=route.next_hop_mac) .
3. Sends a CIRCUIT_SETUP packet toward DST_ID.
4. For subsequent anonymous data packets: sets SRC_ID = CIRCUIT_ID as big-endian uint32 followed by 12 zero bytes, and sets CIRCUIT_ID field to the circuit_id value.

Intermediate node behavior on CIRCUIT_SETUP:

1. Looks up `DST_ID` in routing table to find next hop.
2. Stores (`circuit_id`, `prev=via_mac`, `next=next_hop_mac`) .
3. Forwards the `CIRCUIT_SETUP` toward `DST_ID`.

Destination behavior on `CIRCUIT_SETUP`:

1. Stores (`circuit_id`, `prev=via_mac`, `next=None`) .
2. Does not forward further.

Anonymous forwarding:

A node that receives a `DATA` packet with `CIRCUIT_ID != 0` :

1. Looks up `circuit_id` in its circuit table.
2. If not found: discard silently.
3. If `next_hop_mac` is `None`: deliver `PAYLOAD` locally (this node is the destination).
4. Otherwise: forward the packet to `next_hop_mac` .

Privacy properties:

- Intermediate nodes see only `prev_hop_mac` and `next_hop_mac` for their circuit table entry. They cannot determine the originator or destination.
- The destination sees the anonymized `SRC_ID` (derived from `CIRCUIT_ID`) and cannot determine which physical node sent the message.
- A passive observer on a single link sees only the two adjacent MACs, not the full path.

Circuit teardown: Either endpoint MAY send a `CIRCUIT_TEARDOWN` packet. Each node receiving `CIRCUIT_TEARDOWN` MUST remove the circuit entry. If not the intended destination, the node MUST forward the `CIRCUIT_TEARDOWN` along the circuit.

Circuit lifetime: Circuits persist until explicitly torn down or until all associated routing entries expire. Implementations SHOULD tear down circuits when the session ends.

6.6. HELLO and BYE Messages

HELLO

Purpose: announce presence and discover neighbors.

A node MUST send a `HELLO` broadcast upon joining the mesh. A node MUST send periodic `HELLO` broadcasts at intervals of `HELLO_INTERVAL` (default: 30 seconds).

`HELLO` packet fields:

```
PKT_TYPE  = 0x01
SRC_ID    = this node's Node ID
DST_ID    = broadcast (0xFF * 16)
TTL       = 16
PAYLOAD   = this node's Node ID (16 bytes)
```

Upon receiving a `HELLO` packet:

1. Register the sender in the local node directory with `cost = 1` and `last_seen = now` .
2. Add a direct route: `dst_id = packet SRC_ID` , `next_hop_mac = via_mac` , `cost = 1` .
3. If `SRC_ID` differs from this node's Node ID (i.e., the `HELLO` is not an echo), transmit a `HELLO` reply as a unicast to `via_mac` , so the sender can also register this node.
4. Broadcast a `ROUTE_UPDATE` with the updated table.

Node directory expiration: Nodes not heard from within `STALE_NODE_S` (default: 90 seconds) MUST be removed from the node directory and their direct routes removed from the routing table.

BYE

Purpose: graceful departure notification.

A node SHOULD send a `BYE` broadcast before leaving the mesh to accelerate convergence at remaining nodes.

`BYE` packet fields:

```
PKT_TYPE  = 0x02
SRC_ID    = this node's Node ID
DST_ID    = broadcast (0xFF * 16)
TTL       = 16
PAYLOAD   = empty
```

Upon receiving a `BYE` packet:

1. Remove the sender from the node directory.
2. Remove the direct route to the sender from the routing table.
3. Broadcast a `ROUTE_UPDATE` so other nodes can update their own tables immediately.

6.7. ROUTE_UPDATE Messages

ROUTE_UPDATE packets propagate routing information using a simplified distance-vector algorithm.

ROUTE_UPDATE packet fields:

```
PKT_TYPE   = 0x05
SRC_ID     = this node's Node ID
DST_ID     = broadcast (0xFF * 16)
TTL        = 1 (single-hop propagation ONLY)
PAYLOAD    = JSON array as specified in Section 6.4
```

Processing on receipt:

1. Parse the JSON payload array.
2. For each entry:

```
dst_id = bytes.fromhex(entry["dst_id"])
cost   = int(entry["cost"]) + 1
if cost < TTL_DEFAULT:
    add_route(dst_id, via_mac, cost)
```

3. Do NOT re-broadcast. (TTL=1 enforces single-hop propagation mechanically, but implementations MUST also check and skip re-broadcasting regardless of TTL.)

7. Adapter Abstraction and Channel Selection

7.1. NetworkAdapter Interface

All physical and logical channels implement a common `NetworkAdapter` interface:

Interface `NetworkAdapter`:

```
send(data: bytes) -> bool
    Transmit data bytes on this channel.
    Returns True on success.
    Returns False on collision or channel unavailable.

receive() -> bytes | None
    Return the next received payload, or None if the
    receive buffer is empty.
    MUST NOT block.

is_available() -> bool
    Return True if the channel is ready for operation.
    A channel is available when its handshake is complete
    (for QR_LIGHT) or its socket is open (for TCP/UDP).

get_mac() -> bytes (6 bytes)
    Return the 6-byte link-layer address of this adapter.

get_type() -> AdapterType
    Return the adapter type. Defined values:
    QR_LIGHT      Optical QR channel
    ETHERNET      TCP/IP channel
    WIFI          UDP multicast channel
    VIDEO_STREAM  Video conferencing channel (reserved)

get_cost() -> int
    Return the cost metric for AdapterSelector scoring.
    Default costs:
    QR_LIGHT      100
    ETHERNET      10
    WIFI          20
    VIDEO_STREAM  50
```

7.2. AdapterSelector

AdapterSelector holds a list of registered `NetworkAdapter` instances and selects the best one for each outgoing packet.

Selection policies:

`PREFER_QR` — Always select `QR_LIGHT` if `is_available()`; otherwise fall back to the lowest-cost available adapter.

`PREFER_TCP` — Always select `ETHERNET` if `is_available()`; otherwise fall back to the lowest-cost available adapter.

`LOWEST_COST` — Select the available adapter with the minimum cost. Score function:

```
score(a) = a.get_cost() + 1000 * (1 if not a.is_available() else 0)
```

The adapter with the lowest score is selected.

`FIRST_AVAILABLE` — Select the first adapter in registration order for which `is_available()` returns `True`.

The selector **MUST** skip adapters for which `is_available()` returns `False`. If no adapter is available, the outgoing transmission **MUST** fail and `send()` **MUST** return `False`.

The selection policy **MAY** be changed at runtime without restarting the node.

7.3. Ethernet Adapter (TCP)

The `EthernetAdapter` provides a `NetworkAdapter` over TCP/IP. It uses a persistent server socket for reception and short-lived client connections for transmission.

Design:

Server side: Listens on `host:port`. For each incoming connection, reads one framed message, enqueues it, and closes the connection.

Client side: To transmit to a known peer, opens a TCP connection to `peer_host:peer_port`, sends one framed message, and closes the connection.

Framing:

TCP is a stream protocol without message boundaries. `EthernetAdapter` prefixes each message with a 4-byte big-endian unsigned length header:

```

      0           1           2           3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           LENGTH (32 bits, big-endian)           |
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
|           PAYLOAD (LENGTH bytes)                 |
~                                                     ~
+---+---+---+---+---+---+---+---+---+---+---+---+---+---+---+
```

Maximum message size: 65,535 bytes.

MAC derivation: If no MAC is explicitly provided, `EthernetAdapter` derives a deterministic 6-byte MAC from `MD5("host:port")[:6]`. This ensures distinct MACs for two instances on the same host using different ports.

Port exclusivity: On Windows, `SO_EXCLUSIVEADDRUSE` **MUST** be set on the server socket to prevent two processes from binding the same port. On POSIX systems, `SO_REUSEADDR` is used to allow rebinding after `TIME_WAIT`.

Default listening port: **9000**.

Default cost: **10**.

Socket timeout: **2 seconds** (for receive and server accept).

7.4. WiFi Adapter (UDP Multicast)

The `WifiAdapter` provides a `NetworkAdapter` over IPv4 UDP multicast. All nodes that join the same multicast group receive all messages sent to the group, reflecting the shared-medium broadcast nature of WiFi.

Multicast group: `239.255.60.60` (administratively scoped per RFC 2365)

Default port: `9001`

Multicast TTL: `1` (**MUST NOT** leave the local network segment)

Framing:

Each UDP datagram carries a fixed 8-byte header followed by the payload:

Physical layer exposure:

The optical channel is visible to any observer with a line of sight to the transmitting display. An adversary near the display can record all transmitted frames. Physical shielding of the display is the operator's responsibility in adversarial environments.

Denial of service:

An adversary with physical access to the optical channel can disrupt communication by:

- Projecting conflicting light patterns at the display.
- Physically obstructing the camera.
- Injecting spurious frames.

No cryptographic protection against physical-layer DoS is defined in this specification.

Checksum security:

CRC-16 and CRC-32 are designed for error detection, not for cryptographic integrity. An adversary who can modify frames or packets in transit can compute a valid checksum for the modified content. Applications requiring integrity protection against active adversaries **MUST** implement a cryptographic MAC (e.g., HMAC-SHA256) at Layer 7.

Multicast group access:

The UDP multicast group `239.255.60.60` is administratively scoped and **MUST NOT** be routed beyond the local network segment. However, any host on the local segment can join the group and read all WiFi-channel traffic. End-to-end encryption at Layer 7 is **RECOMMENDED** for sensitive applications that use the WiFi adapter.

NACK replay:

An adversary who can inject NACK frames could cause the transmitter to repeatedly retransmit frames, consuming bandwidth and CPU resources. Rate-limiting NACK processing is **RECOMMENDED** for deployments where the channel is accessible to untrusted parties.

Handshake spoofing:

The MAGIC field (`0x4E51`) provides a simple frame identifier but is not a security mechanism. An adversary can craft valid HandshakeFrames. Implementations that require authenticated session initiation **MUST** implement additional authentication at Layer 7 or use an external key exchange mechanism before initiating the QR-NET handshake.

10. Acknowledgments

The QR-NET Rainbow 1 protocol was designed and implemented as part of the Redes de Computadoras course at the Instituto Tecnológico de Costa Rica (ITCR), advised by Professor Kevin Moraga.

The distance-vector routing design draws from the concepts introduced in [RFC1058] (RIP) and the anonymous forwarding model is inspired by onion routing principles described in the Tor design document.

The multicolor QR encoding approach extends the ISO/IEC 18004 QR Code specification, which the authors gratefully acknowledge as the foundational grid and finder-pattern design.

The Binary Exponential Backoff algorithm is adapted from IEEE 802.3 Ethernet CSMA/CD, used here in the context of an optical shared medium.

11. IANA Considerations

This document has no IANA actions.

The UDP port 9001 used by the WifiAdapter is not registered with IANA for this purpose. Operators **SHOULD** verify that this port is not in use by another service before deploying QR-NET nodes.

The UDP port 9000 used by the EthernetAdapter server is not registered with IANA for this purpose. The same verification applies.

The IPv4 multicast address `239.255.60.60` falls within the administratively scoped block `239.0.0.0/8` defined in [RFC2365]. No IANA registration is required or appropriate for addresses within this block.

12. References

Normative References

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997. <https://www.rfc-editor.org/info/rfc2119>

[RFC2365] Meyer, D., "Administratively Scoped IP Multicast", BCP 23, RFC 2365, DOI 10.17487/RFC2365, July 1998. <https://www.rfc-editor.org/info/rfc2365>

Informative References

[ISO18004] ISO/IEC 18004:2015, "Information technology – Automatic identification and data capture techniques – QR Code bar code symbology specification", Third edition, 2015.

[RFC793] Postel, J., "Transmission Control Protocol", STD 7, RFC 793, DOI 10.17487/RFC0793, September 1981. <https://www.rfc-editor.org/info/rfc793>

[RFC791] Postel, J., "Internet Protocol", STD 5, RFC 791, DOI 10.17487/RFC0791, September 1981. <https://www.rfc-editor.org/info/rfc791>

[RFC1058] Hedrick, C., "Routing Information Protocol", RFC 1058, DOI 10.17487/RFC1058, June 1988. <https://www.rfc-editor.org/info/rfc1058>

[RFC4271] Rekhter, Y., Li, T., and S. Hares, "A Border Gateway Protocol 4 (BGP-4)", RFC 4271, DOI 10.17487/RFC4271, January 2006. <https://www.rfc-editor.org/info/rfc4271>

[RFC8878] Collet, Y. and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type", RFC 8878, DOI 10.17487/RFC8878, February 2021. <https://www.rfc-editor.org/info/rfc8878>

[IEEE8023] IEEE Std 802.3-2022, "IEEE Standard for Ethernet", IEEE, 2022.

13. Authors' Address

Ion Dolanescu

Instituto Tecnológico de Costa Rica
Redes de Computadoras — Ingeniería de Computación
Alajuela, Costa Rica

Ariana Jimenez Paniagua

Instituto Tecnológico de Costa Rica
Redes de Computadoras — Ingeniería de Computación
Alajuela, Costa Rica

Jose Mario Jimenez Vargas

Instituto Tecnológico de Costa Rica
Redes de Computadoras — Ingeniería de Computación
Alajuela, Costa Rica

Email: iodolanescu@estudiantec.cr

GitHub: <https://github.com/IonDola/proyecto-qr>